

Returning Status

Values and control flow

Gregory S. DeLozier, PhD

Breaking Changes

```
value, status = evaluate(ast, environment)
```

Every evaluator result is now a pair.

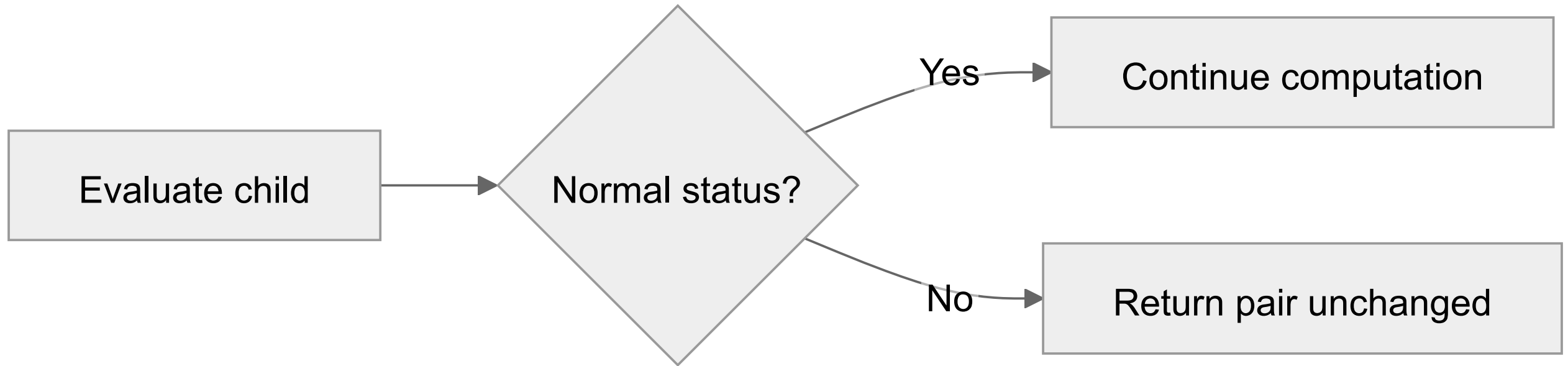
exit and assert become keywords.

Value Is Not Status

Evaluation	Value	Status
2 + 3	5	None
2 > 3	False	None
print(5)	None	None
exit(7)	7	"exit"

False is a normal result.

The Caller Rule



Check status before using the value.

Two Operands, Two Checks

```
left, status = evaluate(ast["left"], environment)
if status is not None:
    return left, status
right, status = evaluate(ast["right"], environment)
if status is not None:
    return right, status
return left + right, None
```

Requesting Exit

Source	Returned pair
<code>exit()</code>	<code>(0, "exit")</code>
<code>exit(2 + 3)</code>	<code>(5, "exit")</code>
<code>exit(exit(7))</code>	<code>(7, "exit")</code>

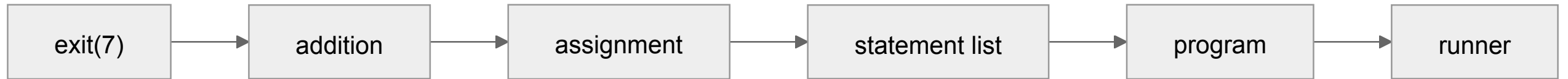
Parentheses required. Integer argument optional.

Where Does Execution Stop?

```
x = 3;  
x = exit(7) + number(input());  
print(x);
```

No input. No addition. No assignment. No print.

Exit Returns Outward



Every arrow carries **(7, "exit")**.

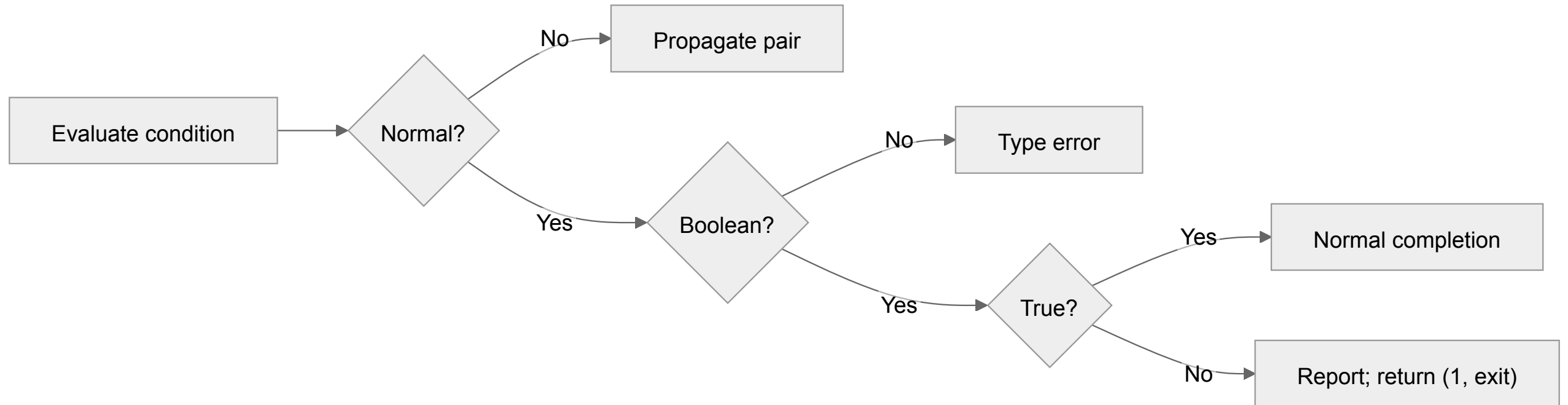
Assertions State Expectations

```
answer = 41;  
assert answer == 42, "Expected 42";  
print("This must not run");
```

Assertion failed: Expected 42

Report here. Return (1, "exit").

Assertion Workflow



The Runner Handles Exit

```
value, status = evaluate(ast, {})  
if status == "exit":  
    sys.exit(value)  
if status is not None:  
    raise ValueError(f"Unhandled status: {status}")
```

Evaluation is testable without terminating the test process.

Skipped Nodes Produce No Status

Expression	Result
<code>false and exit(9)</code>	<code>(False, None)</code>
<code>true or exit(9)</code>	<code>(True, None)</code>
<code>true and exit(9)</code>	<code>(9, "exit")</code>
<code>type(exit(9))</code>	<code>(9, "exit")</code>

Stopping Is Not Undoing

<code>input() + exit(7)</code>	input is consumed
<code>exit(7) + input()</code>	input is skipped

Both return **(7, "exit")**.

Test the Pair and the Effects

```
result, env = evaluate_source("x=3;x=exit(7);x=99")  
assert result == (7, "exit")  
assert env["x"] == 3
```

Exit exactly once. Later work never runs.

Questions for Thought

Why check status before storing an assignment?

Which structure should handle break?

What would recovery from assertion failure require?